Prelucrarea Imaginilor

Prof. Dr. Călin-Adrian POPA

Cursul 10

24 Noiembrie 2025

10 Generative models

- throughout most of this course, we have talked about how to make predictions
- in some form or another, we used deep neural networks to learn mappings from data examples to labels
- this kind of learning is called *discriminative learning*, meaning that we would like to be able to discriminate between photos of cats and photos of dogs
- classifiers and regressors are both examples of discriminative learning
- also, neural networks trained by backpropagation have obtained remarkable performance in discriminative learning on large complicated datasets
- for example, classification accuracies on high-resolution images have gone from useless to human-level in just 5-6 years

10 Generative models

- but there is more to machine learning than just solving discriminative tasks
- for example, given a large dataset, without any labels, we might want to learn a model that concisely captures the characteristics of this data
- given such a model, we could sample synthetic data examples that resemble the distribution of the training data
- for example, given a large corpus of photographs of faces, we might want to be able to generate a new photorealistic image that looks like it might plausibly have come from the same dataset
- this kind of learning is called *generative modeling*

10 Generative models

- until recently, we had no method that could synthesize novel photorealistic images
- but the success of deep neural networks for discriminative learning opened up new possibilities
- one big trend over the last years has been the application of discriminative deep networks to overcome challenges in problems that we do not generally think of as supervised learning problems
- the recurrent neural network language models are one example of using a discriminative network (trained to predict the next character) that, once trained, can act as a generative model

10.1 Generative adversarial networks (GANs)

- in 2014, a breakthrough paper introduced *generative adversarial networks* (*GANs*), a clever new way to leverage the power of discriminative models to obtain good generative models
- at their heart, GANs rely on the idea that a data generator is good if we cannot tell fake data apart from real data
- in statistics, this is called a *two-sample test* – a test to answer the question whether datasets $X = \{x_1, \ldots, x_n\}$ and $X' = \{x'_1, \ldots, x'_n\}$ were drawn from the same distribution
- the main difference between most statistics papers and GANs is that the latter use this idea in a constructive way
- in other words, rather than just training a model to say "these two datasets do not look like they came from the same distribution", they use the two-sample test to provide training signals to a generative model
- this allows us to improve the data generator until it generates something that resembles the real data
- at the very least, it needs to fool the classifier, even if our classifier is a state-of-the-art deep neural network

10.1 Generative adversarial networks (GANs)

- the GAN architecture is illustrated in Figure 1
- as we can see, there are two parts in the GAN architecture – first off, we need a device (say, a deep network, but it really could be anything, such as a game rendering engine) that might potentially be able to generate fake data that looks just like the real data
- if we are dealing with images, this needs to generate images
- if we are dealing with speech, it needs to generate audio sequences, and so on
- we call this the *generator network*

10.1 Generative adversarial networks (GANs)

- the second component is the *discriminator network*
- it attempts to distinguish fake and real data from each other
- both networks are in competition with each other
- the generator network attempts to fool the discriminator network
- at that point, the discriminator network adapts to the new fake data
- this information, in turn, is used to improve the generator network, and so on

10.1 Generative adversarial networks (GANs)

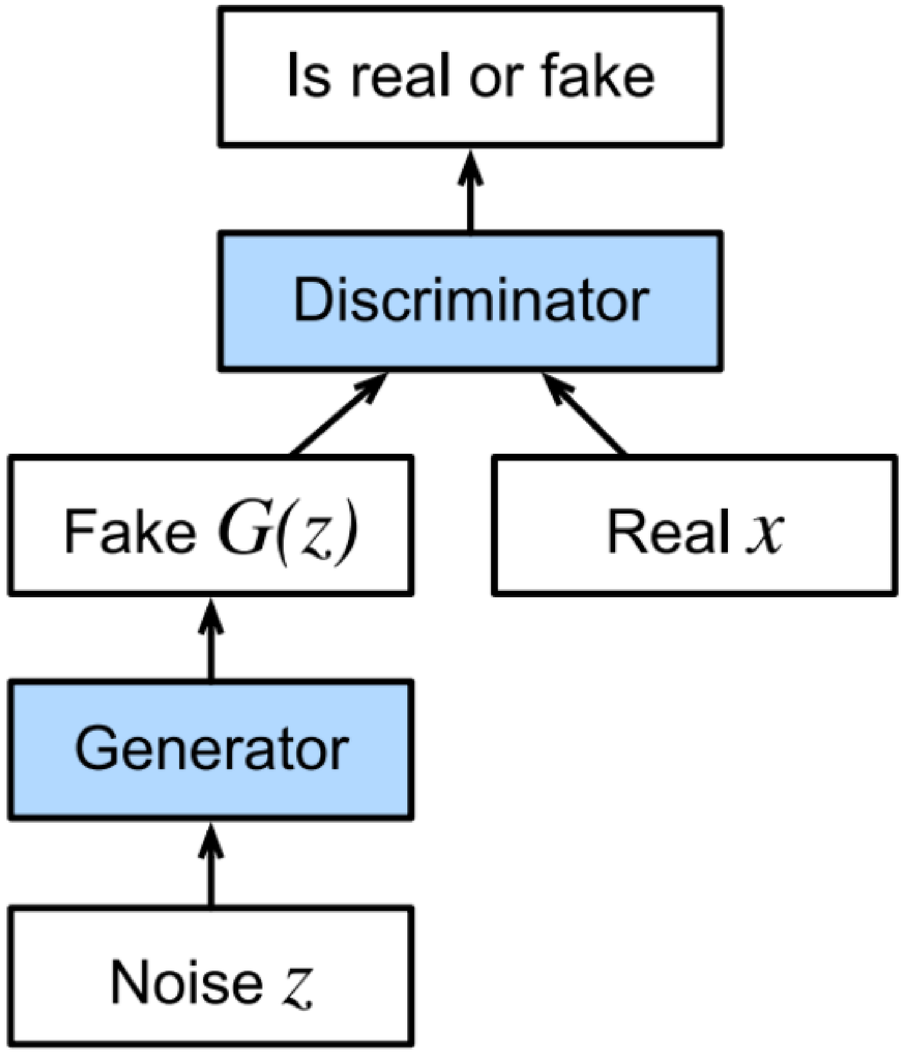

Figure 1: Generative adversarial networks (GANs).

10.1 Generative adversarial networks (GANs)

- the discriminator is a binary classifier to distinguish if the input $\boldsymbol{x}$ is real (from real data) or fake (from the generator)
- typically, the discriminator outputs a scalar prediction $o \in \mathbb{R}$ for input $\boldsymbol{x}$, such as using a fully-connected layer with hidden size 1, and then applies the sigmoid function to obtain the predicted probability $D(\boldsymbol{x}; \theta_d) = 1/(1 + e^{-o})$, where θ_d are the parameters of the discriminator
- assume the label y for the true data is 1 and for the fake data is 0
- we train the discriminator to minimize the cross-entropy loss, i.e.,

$$\min_D \{-y \log D(\boldsymbol{x}) - (1 - y) \log(1 - D(\boldsymbol{x}))\}.$$

- the generator first draws some parameter $\boldsymbol{z} \in \mathbb{R}^d$ from a source of randomness, e.g., a normal distribution $\boldsymbol{z} \sim \mathcal{N}(0, 1)$
- we often call $\boldsymbol{z}$ the *latent variable*

10.1 Generative adversarial networks (GANs)

- it then applies a function $G(\boldsymbol{z}; \theta_g)$, where θ_g are the parameters of the generator, to generate $\boldsymbol{x}' = G(\boldsymbol{z})$
- the goal of the generator is to fool the discriminator to classify $\boldsymbol{x}' = G(\boldsymbol{z})$ as true data, i.e., we want $D(G(\boldsymbol{z})) \approx 1$
- in other words, for a given discriminator D, we update the parameters of the generator G to maximize the cross-entropy loss when $y = 0$, i.e.,

$$\max_G \{-(1-y)\log(1 - D(G(\boldsymbol{z})))\} = \max_G \{-\log(1 - D(G(\boldsymbol{z})))\}.$$

- if the generator does a perfect job, then $D(\boldsymbol{x}') \approx 1$, so the above loss is near 0, which results in the gradients that are too small to make good progress for the discriminator

10.1 Generative adversarial networks (GANs)

- so, commonly, we minimize the following loss:

$$\min_G\{-y\log(D(G(\boldsymbol{z})))\} = \min_G\{-\log(D(G(\boldsymbol{z})))\},$$

  which is just feeding $\boldsymbol{x}' = G(\boldsymbol{z})$ into the discriminator, but giving label $y = 1$
- to sum up, D and G are playing a "minimax" game with the comprehensive objective function:

$$\min_D \max_G\{-E_{\boldsymbol{x}\sim\text{Data}}[\log D(\boldsymbol{x})] - E_{\boldsymbol{z}\sim\text{Noise}}[\log(1 - D(G(\boldsymbol{z})))]\}.$$

- many of the applications of GANs are in the context of images
- the stochastic gradient descent training of GANs follows (k is a hyperparameter used with value $k = 1$ in the experiments):

10.1 Generative adversarial networks (GANs)

for number of training iterations **do**
  for k steps **do**
    Sample minibatch of m noise samples $\{\boldsymbol{z}^{(1)}, \ldots, \boldsymbol{z}^{(m)}\}$ from Noise.
    Sample minibatch of m examples $\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)}\}$ from data generating distribution Data.
    Update the discriminator by stochastic gradient descent:

$$\theta_d \leftarrow \theta_d - \eta \nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^{m} \left[-\log D\left(\boldsymbol{x}^{(i)}\right) - \log\left(1 - D\left(G\left(\boldsymbol{z}^{(i)}\right)\right)\right)\right].$$

  end for
  Sample minibatch of m noise samples $\{\boldsymbol{z}^{(1)}, \ldots, \boldsymbol{z}^{(m)}\}$ from Noise.
  Update the generator by stochastic gradient ascent:

$$\theta_g \leftarrow \theta_g + \eta \nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^{m} -\log\left(1 - D\left(G\left(\boldsymbol{z}^{(i)}\right)\right)\right).$$

end for

10.2 DCGANs

- previous attempts to scale up GANs using CNNs to model images have been unsuccessful
- however, after extensive model exploration, it has been identified a family of architectures that resulted in stable training across a range of datasets and allowed for training higher-resolution and deeper generative models
- core to the approach is adopting and modifying three previous demonstrated changes to CNN architectures

10.2 DCGANs

- the first is the all-convolutional network, which replaces deterministic spatial pooling functions (such as max-pooling) with strided convolutions, allowing the network to learn its own spatial downsampling
- this approach will be used in the generator, allowing it to learn its own spatial upsampling, and also in the discriminator

10.2 DCGANs

- second is the trend towards eliminating fully-connected layers on top of convolutional features
- the strongest example of this is global average pooling, which has been utilized in image classification models
- global average pooling was found to increase model stability, but hurt convergence speed
- a middle ground of directly connecting the highest convolutional features to the input and output, respectively, of the generator and discriminator worked well
- the first layer of the GAN, which takes a uniform noise distribution $\boldsymbol{z}$ as input, could be called fully-connected, as it is just a matrix multiplication, but the result is reshaped into a 4-dimensional tensor and used as the start of the convolutional stack
- for the discriminator, the last convolutional layer is flattened and then fed into a single sigmoid output
- see Figure 2 for a visualization of an example model architecture

10.2 DCGANs

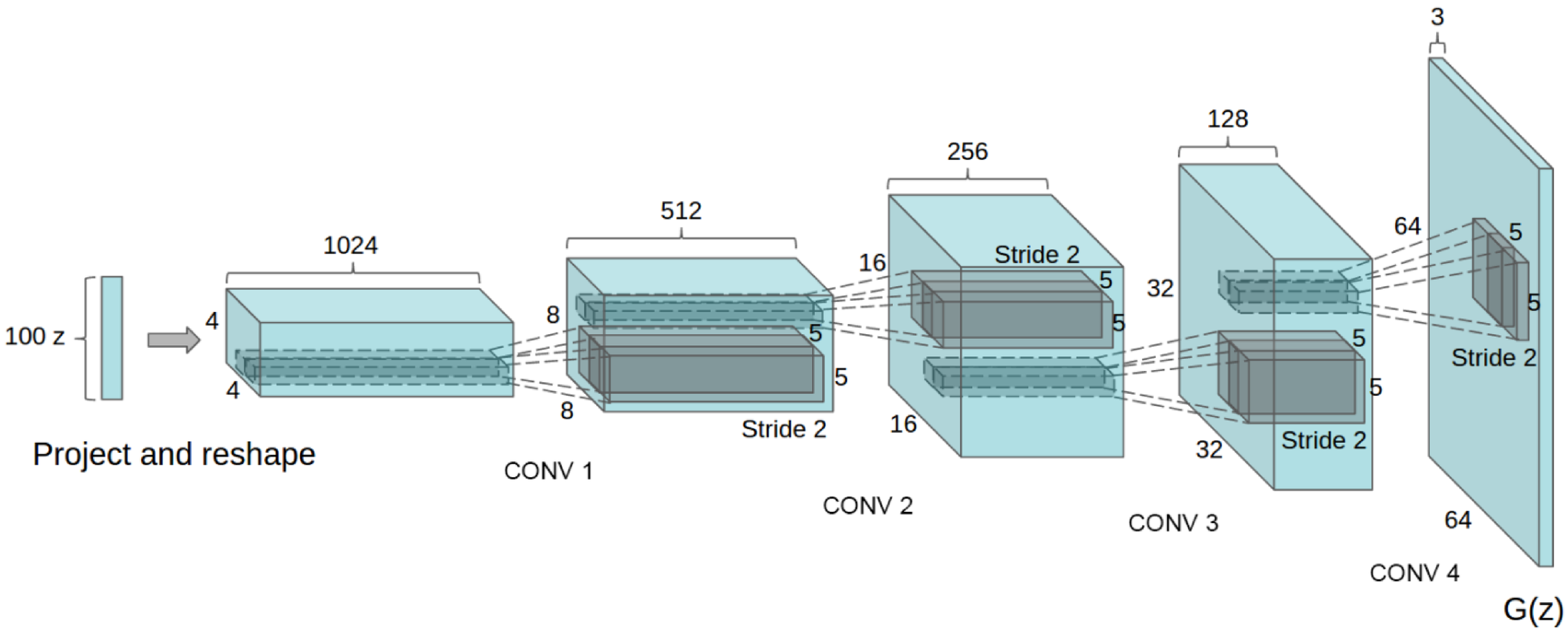

Figure 2: DCGAN generator used for LSUN scene modeling. A 100-dimensional uniform distribution $\boldsymbol{z}$ is projected to a small spatial extent convolutional representation with many feature maps. A series of four transposed convolutions then convert this high-level representation into a 64×64-pixel image. Notably, no fully-connected or pooling layers are used.

10.2 DCGANs

- third is batch normalization, which stabilizes learning by normalizing the input to each unit to have zero mean and unit variance
- this helps deal with training problems that arise due to poor initialization and helps gradient flow in deeper models
- this proved critical to get deep generators to begin learning, preventing the generator from collapsing all samples to a single point, which is a common failure mode observed in GANs
- directly applying batch normalization to all layers, however, resulted in sample oscillation and model instability
- this was avoided by not applying batch normalization to the generator output layer and the discriminator input layer

10.2 DCGANs

- the ReLU activation is used in the generator, with the exception of the output layer, which uses the tanh function
- it has been observed that using a bounded activation allowed the model to learn more quickly to saturate and cover the color space of the training distribution
- within the discriminator, it was found the that Leaky ReLU works well, especially for higher-resolution modeling
- this is in contrast to the original GAN paper, which used the maxout activation

10.2 DCGANs

- to conclude, following are architecture guidelines for stable Deep Convolutional GANs:
  - Replace any pooling layers with strided convolutions (discriminator) and transposed convolutions (generator).
  - Use batch normalization in both the generator and the discriminator.
  - Remove fully-connected hidden layers for deeper architectures.
  - Use ReLU activation in generator for all layers except for the output, which uses tanh.
  - Use LeakyReLU activation in the discriminator for all layers.
- samples from the DCGAN model trained on the LSUN bedrooms dataset containing a little over 3 million training examples can be seen in Figure 3

10.2 DCGANs

Figure 3: Generated bedrooms after five epochs of training on the LSUN bedrooms dataset. There appears to be evidence of visual underfitting via repeated noise textures across multiple samples, such as the base boards of some of the beds.

Vă mulţumesc!